



Pakiet rekrutacyjny JavaReady

Pakiet rekrutacyjny JavaReady

Wstęp

Cześć, tutaj Bartek Kalka z javaready.pl 😊

Widziałem już setki procesów rekrutacyjnych i wiesz co zauważyłem? Większość programistów przygotowuje się do nich... w zupełnie niewłaściwy sposób.

Uczą się na pamięć definicji, zamiast zrozumieć, czego faktycznie szukają rekruterzy.

Zebrałem dla Ciebie ponad 40 **prawdziwych** zadań rekrutacyjnych, które moi podopieczni dostawali podczas rozmów o pracę. To nie są wymyślone ćwiczenia - to autentyczne wyzwania, z którymi zmierzysz się podczas rekrutacji.

Jeden z moich studentów powiedział kiedyś: „Gdybym wcześniej wiedział, o co będą pytać, spędziłbym czas na nauce zupełnie innych rzeczy”. I miał rację.

Potraktuj to kompendium jako mapę drogową do Twojej wymarzonej pracy. Pokażę Ci nie tylko zadania, ale też jakich umiejętności szukają firmy.

Powodzenia!

4 bardzo wartościowe bonusowe materiały (koniecznie sprawdź!):

🎁 Jak 15 osób dostało pierwszą pracę w Javie w 7-10 miesięcy (prawdziwe liczby).

🎁 526+ firm IT przyjmujących juniorów - lista gdzie warto aplikować

🎁 Symulacja rozmowy technicznej - typowe pytania i wzorcowe odpowiedzi

10 typowych błędów - przez które firmy Cię nie wybierają

Spis treści

Pakiet rekrutacyjny JavaReady

Wstęp

4 bardzo wartościowe bonusowe materiały (koniecznie sprawdź!):

Spis treści

1. Algorytmy i struktury danych

- 1.1. Łączenie i sortowanie list
- 1.2. Analiza danych giełdowych
- 1.3. Kompresja tekstu algorytmem RLE
- 1.4. Obliczanie czasu między godzinami
- 1.5. Wyszukiwanie słowa z największą liczbą powtarzających się liter
- 1.6. Liczby bez zera w zapisie dziesiętnym
- 1.7. Walidacja numeru telefonu
- 1.8. Silnia dużych liczb
- 1.9. Magiczny kwadrat
- 1.10. Analiza kierunku lotu samolotu

2. Implementacja klas i interfejsów

- 2.1. Klasa Wall implementująca interfejs Structure
- 2.2. Implementacja klas Student i Nauczyciel
- 2.3. Implementacja prostego "store"
- 2.4. System zarządzania biblioteką książek
- 2.5. Znajdowanie największego samochodu (klasa Car)
- 2.6. Rekurencyjne obliczanie ciągu liczbowego

3. Spring i Spring Boot

- 3.1. CRUD z wykorzystaniem Spring Boot
- 3.2. Aplikacja z kursami walut NBP
- 3.3. Obsługa API pogodowego
- 3.4. Aplikacja do zarządzania biblioteką książek
- 3.5. URL Shortener

4. REST API i aplikacje webowe

- 4.1. Implementacja GitHub API Client
- 4.2. Serwer TCP/IP z dostępem do bazy danych

5. Bazy danych i SQL

5.1. Zaawansowane zapytanie SQL

5.2. Indeksy w bazach danych

5.3. Projektowanie schematu bazy danych dla sklepu internetowego

5.4. Problem N+1 zapytań

5.5. Relacje między tabelami

6. Wielowątkowość i programowanie współbieżne

6.1. Synchronizacja wątków

6.2. Implementacja puli wątków

6.3. Zabezpieczenie URL Shortenera przed problemami współbieżności

6.4. Implementacja producenta-konsumenta

6.5. Zarządzanie zadaniami z priorytetami

7. Typowe pytania teoretyczne

7.1. Pytania dotyczące Java

7.2. Pytania dotyczące Spring

7.3. Pytania dotyczące baz danych

7.4. Pytania dotyczące projektowania aplikacji

8. Zadania praktyczne na cały projekt

8.1. Implementacja biblioteki książek

8.2. Aplikacja do zarządzania kampaniami reklamowymi

8.3. Aplikacja NBP (National Bank of Poland)

8.4. Aplikacja do skracania URL-i

8.5. System zarządzania zadaniami (Todo list)

9. Relacje z prawdziwych rozmów rekrutacyjnych

9.1. Rozmowa na stanowisko Junior Java Developer

9.2. Rozmowa techniczna z algorytmami

9.3. TDD i Spock

9.4. Rekrutacja Spring Boot z API zewnętrznym

9.5. Pytania teoretyczne z rozmowy rekrutacyjnej

Podsumowanie

1. Algorytmy i struktury danych

1.1. Łączenie i sortowanie list

Treść zadania: Masz dwie posortowane listy jednokierunkowe `list1` i `list2`. Połącz je w jedną posortowaną listę poprzez scalenie węzłów obu list. Zwróć głowę połączonej listy.

Przykład:

- Wejście: list1 = [1,2,4], list2 = [1,3,4]
- Wyjście: [1,1,2,3,4,4]

Ograniczenia:

- Liczba węzłów w obu listach jest z przedziału [0, 50]
- Wartości węzłów są z przedziału [-100, 100]
- Obie listy są posortowane w porządku rosnącym

1.2. Analiza danych giełdowych

Treść zadania: Napisz funkcję, która przyjmie tablicę liczb całkowitych reprezentujących cenę akcji w kolejnych dniach. Znajdź maksymalny możliwy zysk, jaki można osiągnąć kupując akcje jednego dnia i sprzedając je innego dnia, gdzie dzień sprzedaży jest po dniu zakupu.

Przykłady:

- Dla tablicy [44, 30, 24, 32, 35, 30, 40, 38, 15] funkcja powinna zwrócić 16 (kupno przy cenie 24, sprzedaż przy 40)
- Dla tablicy [10, 9, 8, 2] funkcja powinna zwrócić -1 (nie można osiągnąć zysku)

1.3. Kompresja tekstu algorytmem RLE

Treść zadania: Zaimplementuj funkcję, która kompresuje podany ciąg znaków używając algorytmu kodowania RLE (Run-length encoding). Algorytm działa tak, że bierze wystąpienia każdego powtarzającego się znaku i zamienia je na liczbę wystąpień oraz jeden znak z powtarzającej się sekwencji.

Przykłady:

- "wwwggopp" powinno zwrócić "3w2g1o2p"
- "aabbccde" powinno zwrócić "2a2b1c1d1e"

1.4. Obliczanie czasu między godzinami

Treść zadania: Napisz funkcję, która przyjmuje parametr w postaci dwóch godzin (każda poprawnie sformatowana z dwukropkiem i oznaczeniem am/pm)

oddzielonych myślnikiem i zwraca całkowitą liczbę minut między tymi dwoma godzinami. Czas będzie podany w 12-godzinny formacie zegara.

Przykłady:

- "9:00am-10:00am" powinno zwrócić 60
- "1:00pm-11:00am" powinno zwrócić 1320
- "12:30pm-12:00am" powinno zwrócić 690

1.5. Wyszukiwanie słowa z największą liczbą powtarzających się liter

Treść zadania: Napisz funkcję, która przyjmuje ciąg znaków i zwraca pierwsze słowo z największą liczbą powtarzających się liter.

Przykłady:

- "Today, is the greatest day ever!" powinno zwrócić "greatest" (ponieważ ma 2 'e' i 2 't' i pojawia się wcześniej niż "ever", które też ma 2 'e')
- "Hello apple pie" powinno zwrócić "Hello" (ma 2 'l')

1.6. Liczby bez zera w zapisie dziesiętnym

Treść zadania: Znajdź dwie dodatnie liczby całkowite (A i B), których suma daje określoną liczbę N, a żadna z tych liczb nie może mieć zera w zapisie dziesiętnym.

1.7. Walidacja numeru telefonu

Treść zadania: Stwórz aplikację do walidacji numerów telefonów. Aplikacja powinna przyjmować numer telefonu jako parametr wejściowy i sprawdzać, czy jest on poprawny zgodnie z określonymi kryteriami:

- Składa się z dokładnie 9 cyfr
- Zawiera tylko cyfry od 0 do 9

Dodatkowe punkty za:

- Obsługę różnych formatów numerów telefonów (np. uwzględnienie myślników, nawiasów czy innych separatorów)

- Zaimplementowanie prostego interfejsu użytkownika

1.8. Silnia dużych liczb

Treść zadania: Napisz algorytm obliczający silnię dużych liczb bez używania klasy `BigDecimal`. Silnia liczby n to iloczyn wszystkich liczb naturalnych od 1 do n .

1.9. Magiczny kwadrat

Treść zadania: Napisz algorytm, który sprawdza czy dana tablica dwuwymiarowa jest magicznym kwadratem. Magiczny kwadrat to tablica $n \times n$ wypełniona liczbami $1, 2, \dots, n^2$, w której sumy liczb we wszystkich wierszach, kolumnach i przekątnych są równe.

1.10. Analiza kierunku lotu samolotu

Treść zadania: Napisz algorytm, który na podstawie podanych stopni określających kierunek lotu samolotu w różnych momentach określi, czy samolot leciał prosto, skręcał w prawo czy w lewo.

2. Implementacja klas i interfejsów

2.1. Klasa `Wall` implementująca interfejs `Structure`

Treść zadania: Zanalizuj kod i zaimplementuj metody `findBlockByColor`, `findBlocksByMaterial` i `count` w klasie `Wall`. Postaraj się unikać powielania kodu i umieść całą logikę w klasie `Wall`.

```
interface Structure {
    // Zwraca dowolny element o podanym kolorze
    Optional<Block> findBlockByColor(String color);

    // Zwraca wszystkie elementy z danego materiału
    List<Block> findBlocksByMaterial(String material);

    // Zwraca liczbę wszystkich elementów tworzących strukturę
    int count();
}
```

```

public class Wall implements Structure {
    private List<Block> blocks;

    @Override
    public Optional<Block> findBlockByColor(String color) {
        // Implementacja metody
        return null;
    }

    @Override
    public List<Block> findBlocksByMaterial(String material)
    {
        // Implementacja metody
        return null;
    }

    @Override
    public int count() {
        // Implementacja metody
        return 0;
    }
}

interface Block {
    String getColor();
    String getMaterial();
}

interface CompositeBlock extends Block {
    List<Block> getBlocks();
}

```

2.2. Implementacja klas Student i Nauczyciel

Treść zadania: Utwórz dwie klasy: Student oraz Nauczyciel. Między nimi ma zachodzić relacja wiele do wielu (nauczyciel może mieć wielu studentów, student może mieć wielu nauczycieli).

Nauczyciel powinien mieć pola: imię, nazwisko, wiek, email oraz przedmiot.

Student powinien mieć pola: imię, nazwisko, wiek, email oraz kierunek.

Wymagania:

1. Obie klasy można tworzyć, usuwać i edytować. Dane powinny być walidowane: poprawny email, imię dłuższe od dwóch liter, wiek > 18.
2. Powinna być możliwość przypisania nauczyciela do studenta oraz usunięcie nauczyciela studentowi jak i na odwrót.
3. Powinna być możliwość wyświetlenia wszystkich studentów oraz wszystkich nauczycieli (dwa endpointy, możliwość stronicowania i sortowania).
4. Dane można filtrować: wyszukać wszystkich studentów danego nauczyciela i odwrotnie.
5. Studentów oraz nauczycieli można wyszukiwać po imieniu i nazwisku.

2.3. Implementacja prostego "store"

Treść zadania: Zaimplementuj prostą klasę Store, która pozwala na przechowywanie danych w postaci klucz-wartość oraz wykonywanie podstawowych operacji na tych danych.

Wymagania:

1. Klasa Store powinna zawierać:
 - Prywatne pole typu `HashMap<String, String>` do przechowywania danych
 - Metodę `set(String key, String value)`, która dodaje nowy klucz-wartość
 - Metodę `get(String key)`, która zwraca wartość dla danego klucza
 - Metodę `remove(String key)`, która usuwa wartość dla danego klucza
 - Metodę `keysStartingWith(String prefix)`, która zwraca zbiór kluczy zaczynających się od danego prefiksu
2. Dodatkowa funkcjonalność:

- Metoda `snapshot()`, która zwraca kopię stanu store'a w postaci nowej instancji klasy `Store`
- Aktualizacja struktury przechowywania danych, tak aby każdy wpis zawierał numer wersji

2.4. System zarządzania biblioteką książek

Treść zadania: Zaprojektuj i zaimplementuj system zarządzania biblioteką książek. Twój system powinien umożliwiać:

1. Dodawanie nowych książek do kolekcji
2. Wypożyczanie i zwracanie książek przez czytelników
3. Sprawdzanie dostępności oraz lokalizacji książek w bibliotece
4. Zarządzanie informacjami o czytelnikach, takimi jak wypożyczone książki i terminy zwrotu

Przykładowy interfejs:

```
public interface LibrarySystem {
    void addBook(Book book);
    void borrowBook(Book book, Reader reader);
    void returnBook(Book book);
    boolean checkAvailability(Book book);
    List<String> locateBook(Book book);
    void manageReader(Reader reader);
}
```

2.5. Znajdowanie największego samochodu (klasa `Car`)

Treść zadania: Utwórz statyczną metodę w klasie `Car`, która przyjmuje tablicę obiektów `Car` i zwraca największy element z tej tablicy. Samochód jest "większy" od drugiego, gdy jego maksymalna prędkość jest większa. Jeśli maksymalne prędkości są równe, większy jest ten samochód, który ma większe przyspieszenie.

Jeśli tablica wejściowa jest pusta, rzuć `IllegalStateException` z komunikatem "Bad array size". Jeśli tablica wejściowa jest null, rzuć `RuntimeException` z

komunikatem "Input array should not be null".

```
public class Car {
    public static Car getMax(Car[] cars) {
        // kod tutaj
    }

    int maxSpeed;
    int acceleration;

    public Car(int maxSpeed, int acceleration) {
        this.maxSpeed = maxSpeed;
        this.acceleration = acceleration;
    }

    @Override
    public String toString() {
        return "Car with maxSpeed=" + maxSpeed + " and acceleration=" + acceleration;
    }
}
```

2.6. Rekurencyjne obliczanie ciągu liczbowego

Treść zadania: Napisz metodę w Javie, która oblicza element ciągu, gdzie każdy element jest sumą 3 poprzednich. Pierwsze trzy elementy tego ciągu to 0, 1, 2. Zatem to ciąg {0, 1, 2, 3, 6, 11, 20, 37, 68, ...}.

```
public int calculate(int index) {
    // KOD TUTAJ
}
```

3. Spring i Spring Boot

3.1. CRUD z wykorzystaniem Spring Boot

Treść zadania: Stwórz aplikację CRUD z wykorzystaniem Spring Boot, która umożliwi:

- Dodawanie, edytowanie, usuwanie i listowanie obiektów
- Walidację danych
- Wykorzystanie bazy H2 startowanej w pamięci
- Prosty frontend do edycji aplikacji (możesz użyć dowolnego frameworka JS)

Obiekt kampanii powinien zawierać następujące pola:

- Nazwa kampanii (obowiązkowe)
- Słowa kluczowe (obowiązkowe, z typeahead)
- Kwota oferty (obowiązkowa, minimalna kwota)
- Fundusz kampanii (obowiązkowy)
- Status (włączony/wyłączony - obowiązkowy)
- Miasto (wybór z rozwijanej listy)
- Promień (obowiązkowy, w kilometrach)

3.2. Aplikacja z kursami walut NBP

Treść zadania: Przygotuj prostą aplikację mikroserwisową z wykorzystaniem Spring i Spring Boot 2.X, która będzie komunikować się z API NBP (Narodowego Banku Polskiego).

Przypadek użycia 1:

Serwis powinien umożliwiać pobieranie kursu sprzedaży waluty w PLN dla podanej waluty i daty publikacji tabeli kursów przez NBP. Aby zmaksymalizować wydajność pobierania tych samych danych, zebrane kursy powinny być buforowane w lokalnej bazie danych (np. H2). Mikroserwis może zwrócić dane klientowi w odpowiedzi tylko wtedy, gdy wszystkie dane z NBP zostały najpierw zapisane w lokalnej bazie danych.

Przypadek użycia 2:

Serwis powinien umożliwiać obliczenie całkowitego kosztu zakupu w PLN dla podanej listy walut obcych dla danej daty. Żądanie będzie zawierać listę walut

obcych w formacie trzyliterowego kodu waluty (format ISO 4217). Całkowity koszt w PLN powinien być przeliczany przy użyciu średniego kursu walut.

3.3. Obsługa API pogodowego

Treść zadania: Stwórz aplikację serwerową z wykorzystaniem Spring Boot, która będzie serwować użytkownikowi historyczne (z minionego tygodnia) dane meteorologiczne z publicznego API (<https://open-meteo.com/>).

Serwer powinien wystawiać endpoint akceptujący długość i szerokość geograficzną. Zwracane informacje muszą zawierać średnią ilość opadów danego dnia oraz czas wschodu/zachodu słońca.

Aplikacja powinna każdorazowo zapisywać w relacyjnej bazie danych czas oraz parametry wywołania endpointu.

3.4. Aplikacja do zarządzania biblioteką książek

Treść zadania: Stwórz aplikację w Spring Boot, która umożliwi zarządzanie biblioteką książek:

- Dodawanie książek do systemu
- Wypożyczanie książek
- Zwrot książek
- Wyszukiwanie książek po różnych kryteriach
- Generowanie raportów o wypożyczeniach

Zastosuj zasady RESTful API i zadбай o odpowiednią walidację danych.

3.5. URL Shortener

Treść zadania: Zaimplementuj mikroserwis do skracania adresów URL z wykorzystaniem Spring Boot.

Funkcjonalność powinna obejmować:

- Skracanie długich adresów URL
- Przekierowanie z krótkiego URL na oryginalny adres
- Zabezpieczenie przed kolizjami skróconych URL-i

- Zliczanie liczby przekierowań dla każdego skróconego URL-a

Zastanów się nad obsługą wielowątkowości w swojej implementacji.

4. REST API i aplikacje webowe

4.1. Implementacja GitHub API Client

Treść zadania: Zaimplementuj klienta API dla GitHub. Aplikacja powinna spełniać następujące wymagania:

1. Jako konsument API, po podaniu nazwy użytkownika i nagłówka "Accept: application/json", chciałbym zobaczyć listę wszystkich jego repozytoriów GitHub, które nie są forkami. Informacje, które chcę otrzymać w odpowiedzi to:
 - Nazwa repozytorium
 - Login właściciela
 - Dla każdej gałęzi jej nazwa i SHA ostatniego commita
2. Jako konsument API, po podaniu nieistniejącego użytkownika GitHub, chciałbym otrzymać odpowiedź 404 w formacie:

```
{
  "status": ${responseCode},
  "Message": ${whyHasItHappened}
}
```

3. Jako konsument API, po podaniu nagłówka "Accept: application/xml", chciałbym otrzymać odpowiedź 406 w formacie:

```
{
  "status": ${responseCode},
  "Message": ${whyHasItHappened}
}
```

Użyj <https://developer.github.com/v3> jako API backendowego.

4.2. Serwer TCP/IP z dostępem do bazy danych

Treść zadania: Napisz w języku Java prosty serwer TCP/IP, który otworzy Socket i zacznie nasłuchiwać przychodzących danych na wybranym porcie.

Po odebraniu komunikatu (klient powinien wysłać id użytkownika), serwer pobierze wszystkie pojazdy i oferty ubezpieczeniowe dla danego użytkownika oraz odeśle je w odpowiedzi. Do odczytu danych z bazy użyj czystego JDBC.

Struktura bazy danych:

users (tabela z użytkownikami)

- id bigint PRIMARY KEY NOT NULL
- nick text NOT NULL
- login text NOT NULL
- password text NOT NULL
- ... (pozostałe dane o użytkowniku)
- insert_time timestamp NOT NULL DEFAULT now()

vehicles (tabela z pojazdami użytkowników)

- id bigint PRIMARY KEY NOT NULL
- login text NOT NULL REFERENCES(users.login)
- brand text NOT NULL (marka pojazdu)
- model text NOT NULL (model pojazdu)
- ... (pozostałe dane o pojeździe)
- insert_time timestamp NOT NULL

insurance_offers (tabela zawierająca oferty ubezpieczenia OC dla danego pojazdu)

- id bigint PRIMARY KEY NOT NULL
- vehicle_id bigint NOT NULL REFERENCES (vehicles.id)
- insurer text NOT NULL (nazwa towarzystwa ubezpieczeniowego)
- price float NOT NULL
- ... (pozostałe dane o ofercie ubezpieczeniowej)
- insert_time timestamp NOT NULL

Dodatkowe pytania:

- Co można poprawić w strukturze bazy?
- Jak po stronie serwera można zweryfikować czy nadawca komunikatu jest właścicielem danego konta?
- Czy zaproponowane rozwiązanie można zoptymalizować, aby obsłużyć jednocześnie komunikaty od kilku klientów?

5. Bazy danych i SQL

5.1. Zaawansowane zapytanie SQL

Treść zadania: Napisz zaawansowane zapytanie SQL do dwóch rozbudowanych tabel z danymi bankowymi, które uwzględnia:

- Joiny
- Sortowanie
- Grupowanie
- Limitowanie wyników
- Filtrowanie (wyniki większe od jakiejś wartości)

5.2. Indeksy w bazach danych

Treść zadania: Wyjaśnij czym są indeksy w bazach danych i do czego służą. Podaj przykłady, kiedy warto je stosować, a kiedy mogą pogorszyć wydajność.

5.3. Projektowanie schematu bazy danych dla sklepu internetowego

Treść zadania: Zaprojektuj schemat bazy danych dla prostego sklepu internetowego. Schema powinien uwzględniać:

- Produkty (z kategoriami i podkategoriami)
- Użytkowników
- Zamówienia
- Koszyk zakupowy
- Opinie o produktach

- Promocje i rabaty

Narysuj diagram ERD (Entity Relationship Diagram) i napisz skrypt SQL tworzący wszystkie tabele z odpowiednimi relacjami.

5.4. Problem N+1 zapytań

Treść zadania: Wyjaśnij na czym polega problem N+1 zapytań w kontekście ORM (np. Hibernate). Zaproponuj rozwiązania tego problemu i zaimplementuj przykład w Javie z wykorzystaniem Spring Data JPA.

5.5. Relacje między tabelami

Treść zadania: Wyjaśnij różnice między relacjami typu one-to-one, one-to-many i many-to-many. Podaj przykłady każdej z nich w kontekście rzeczywistej aplikacji i zaimplementuj je w Javie z wykorzystaniem JPA.

6. Wielowątkowość i programowanie współbieżne

6.1. Synchronizacja wątków

Treść zadania: Napisz klasę implementującą prosty licznik, który będzie działał poprawnie w środowisku wielowątkowym. Zaimplementuj dwie metody:

`increment()` i `getValue()`. Metoda `increment()` powinna zwiększać wartość licznika o 1, a `getValue()` zwracać aktualną wartość licznika.

Zaimplementuj dwie wersje:

1. Wykorzystującą słowo kluczowe `synchronized`
2. Wykorzystującą klasy z pakietu `java.util.concurrent`

6.2. Implementacja puli wątków

Treść zadania: Zaimplementuj prostą pulę wątków, która będzie wykonywać zadania dodawane do kolejki. Pula powinna mieć metody:

- `submit(Runnable task)` - dodaje zadanie do kolejki
- `shutdown()` - zatrzymuje pulę wątków
- `awaitTermination(long timeout, TimeUnit unit)` - czeka na zakończenie wszystkich zadań lub upływ czasu

6.3. Zabezpieczenie URL Shortenera przed problemami współbieżności

Treść zadania: Rozszerz zadanie z URL Shortenerem tak, aby było bezpieczne w środowisku wielowątkowym. Zastanów się nad potencjalnymi problemami związanymi z konkurencyjnym dostępem do danych i zaproponuj rozwiązania.

6.4. Implementacja producenta-konsumenta

Treść zadania: Zaimplementuj wzorzec producent-konsument. Producent generuje liczby i umieszcza je w buforze o ograniczonej pojemności, a konsument pobiera je z bufora. Zarówno producent, jak i konsument działają w osobnych wątkach. Zadbaj o poprawną synchronizację dostępu do współdzielonego bufora.

6.5. Zarządzanie zadaniami z priorytetami

Treść zadania: Proces wykonuje zadania z kolejki. Każde zadanie ma określony priorytet i taki sam czas trwania. Proces zawsze uruchamia zadanie o najwyższym priorytecie. Ciągłe dodawane są nowe zadania o losowych priorytetach. Jak można zmodyfikować tę metodę wykonywania zadań, aby priorytety nadal były uwzględniane, ale nie było niebezpieczeństwa, że niektóre zadania będą czekać w nieskończoność z powodu dużego ruchu przychodzących zadań o wysokim priorytecie?

7. Typowe pytania teoretyczne

7.1. Pytania dotyczące Java

1. Jaka jest różnica między interfejsem a klasą abstrakcyjną?
2. Wyjaśnij czym są słowa kluczowe `final`, `finally` i `finalize`.
3. Czym jest JVM, JRE i JDK?
4. Jaka jest różnica między `ArrayList` a `LinkedList`?
5. Czym są kolekcje w Javie i jakie znasz ich rodzaje?
6. Wyjaśnij koncepcję dziedziczenia w Javie.
7. Co to jest polimorfizm?
8. Czym jest enkapsulacja?

9. Jaka jest różnica między przeciążaniem (overloading) a nadpisywaniem (overriding) metod?
10. Co to są wyjątki? Jaka jest różnica między wyjątkiem typu checked i unchecked?
11. Czym jest Garbage Collector i jak działa?
12. Jaka jest różnica między typami prymitywnymi a ich wrapperami?
13. Wyjaśnij koncepcję interfejsu funkcyjnego i wyrażeń lambda w Javie 8.
14. Co to jest Stream API?
15. Czym jest immutability (niezmienność) obiektów i jak ją zapewnić?

7.2. Pytania dotyczące Spring

1. Czym jest Spring Framework?
2. Co to jest IoC (Inversion of Control) i DI (Dependency Injection)?
3. Czym jest kontener Spring?
4. Jaka jest różnica między zasięgami (scope) beanów w Springu (singleton, prototype, request, session, global session)?
5. Co to jest AOP (Aspect Oriented Programming)?
6. Jakie znasz sposoby wstrzykiwania zależności w Springu?
7. Jakie są stereotypy w Springu i do czego służą?
8. Co to jest Spring Boot i jakie daje korzyści?
9. Czym jest Spring Data?
10. Co to jest Spring Security i jak z niego korzystać?

7.3. Pytania dotyczące baz danych

1. Czym różnią się bazy danych SQL od NoSQL?
2. Co to jest transakcja i jakie ma właściwości (ACID)?
3. Co to jest normalizacja bazy danych?
4. Czym jest klucz główny i klucz obcy?

5. Co to jest indeks w bazie danych?
6. Wyjaśnij różnicę między INNER JOIN a LEFT/RIGHT JOIN.
7. Co to jest ORM (Object-Relational Mapping)?
8. Czym jest Hibernate i JPA?
9. Jak działa mechanizm zarządzania transakcjami w JPA?
10. Co to jest problem N+1 zapytań i jak go rozwiązać?

7.4. Pytania dotyczące projektowania aplikacji

1. Wyjaśnij zasady SOLID.
2. Czym są wzorce projektowe i jakie znasz?
3. Czym jest REST API i jakie ma zasady?
4. Jaka jest różnica między REST a SOAP?
5. Co to jest mikroustuga i jakie są zalety architektury mikroserwisowej?
6. Czym jest konteneryzacja i jak działa Docker?
7. Co to jest CI/CD (Continuous Integration/Continuous Deployment)?
8. Jakie znasz systemy kontroli wersji?
9. Wyjaśnij koncepcję TDD (Test-Driven Development).
10. Co to jest DDD (Domain-Driven Design)?

8. Zadania praktyczne na cały projekt

Wiesz co zawsze mnie zaskakuje? To, jak wielu początkujących programistów myśli, że projekty rekrutacyjne to takie "bajery" do zapomnienia.

A prawda jest taka, że firmy wcale nie wymyślają tych zadań dla zabawy. One naprawdę odzwierciedlają codzienną pracę, która Cię czeka.

8.1. Implementacja biblioteki książek

Kiedys sam dostałem takie zadanie i pamiętam, jak myślałem: "Po co ta cała złożoność? Przecież to tylko rekrutacja."

Treść zadania: Zaprojektuj system zarządzania biblioteką książek z możliwością dodawania, wypożyczania i zwracania książek, plus zarządzanie informacjami o czytelnikach.

Co ciekawe, po kilku miesiącach w nowej pracy robiłem... prawie identyczny system dla klienta.

Wymagania funkcjonalne:

- Dodawanie, edycja i usuwanie książek
- Rejestracja czytelników
- Wypożyczanie i zwracanie książek
- Wyszukiwanie książek po tytule, autorze, kategorii
- Powiadomienia o przekroczeniu terminu zwrotu

To nie są wydumane funkcje. To dokładnie to, czego potrzebuje prawdziwa biblioteka.

Wymagania techniczne:

- Spring Boot
- Baza danych relacyjna
- REST API
- Testy jednostkowe i integracyjne
- Dokumentacja Swagger

8.2. Aplikacja do zarządzania kampaniami reklamowymi

Jeden z moich studentów narzekał: "To zadanie jest bez sensu, przecież nikt takich kampanii nie robi". Trzy miesiące później dostał pracę... w firmie marketingowej.

Treść zadania: Zaimplementuj aplikację do zarządzania kampaniami reklamowymi, z backendem w Spring Boot i prostym frontendem.

Funkcjonalności:

- Tworzenie, edycja i usuwanie kampanii
- Dane kampanii: nazwa, słowa kluczowe, kwota, fundusz, status, miasto, promień
- Walidacja danych
- Historia zmian
- Podstawowe raporty

Zwróć uwagę - te funkcjonalności to niemal dokładna kopia tego, co znajdziesz w Google Ads czy Facebook Ads.

8.3. Aplikacja NBP (National Bank of Poland)

Ten przykład szczególnie lubię, bo pokazuje, jak firmy testują Twoją umiejętność pracy z zewnętrznym API.

Treść zadania: Stwórz mikroservis komunikujący się z API Narodowego Banku Polskiego, który pobiera, zapisuje i udostępnia dane o kursach walut.

Pamiętasz, jak uczyli Cię o korzystaniu z zewnętrznych API? Tutaj dokładnie o to chodzi - o sprawdzenie, czy umiesz:

- Pobierać dane z API NBP
- Zapisywać je w bazie danych
- Udostępniać przez własne API
- Robić obliczenia na tych danych
- Generować wykresy historyczne

To naprawdę przydatne umiejętności, których będziesz używać codziennie.

8.4. Aplikacja do skracania URL-i

Zawsze się uśmiecham, gdy widzę to zadanie. Wiele osób myśli: "Co za banalny projekt, przecież to tylko zamiana długiego URL-a na krótki".

A potem zaczynają i odkrywają, ile tam jest niuansów...

Treść zadania: Zaimplementuj serwis do skracania długich URL-i, generujący krótkie identyfikatory i umożliwiający przekierowania.

To zadanie sprawdza, czy rozumiesz:

- Unikalność identyfikatorów (jak uniknąć kolizji?)
- Wydajność (co jeśli nagle dostaniesz milion zapytań na minutę?)
- Bezpieczeństwo (co jeśli ktoś będzie próbował zgadywać URL-e?)
- Trwałość danych (co jeśli serwer się wyłączy?)

Te same problemy będziesz rozwiązywać w "prawdziwych" projektach.

8.5. System zarządzania zadaniami (Todo list)

"Znowu todo list? Przecież to takie oklepane" - pomyślisz.

I będziesz mieć rację... częściowo.

Treść zadania: Zaprojektuj aplikację do zarządzania zadaniami (todo list), umożliwiającą tworzenie, edytowanie i usuwanie zadań, oraz organizowanie ich w projekty i kategorie.

Todo list to jak "Hello World" aplikacji webowych. Ale nie daj się zwieść prostocie.

Dobra aplikacja todo musi obsługiwać:

- Rejestrację i logowanie
- CRUD dla zadań
- Strukturę projektów i kategorii
- Priorytety i terminy
- Wyszukiwanie i filtrowanie
- Powiadomienia

To tak naprawdę mini-wersja Jiry czy Asany, które są używane przez miliony firm.

9. Relacje z prawdziwych rozmów rekrutacyjnych

Kilka miesięcy temu poprosiłem moich studentów, którzy przeszli przez rekrutację, żeby opisali swoje doświadczenia.

Te relacje są bezcenne, bo pokazują, jak wygląda proces "od kuchni".

9.1. Rozmowa na stanowisko Junior Java Developer

"Byłem na rozmowie na juniora. Pierwsza część to pytania o programowanie obiektowe:

- Różnica między interfejsem a klasą abstrakcyjną
- Kontrakt equals i hashCode
- Optional w Java
- Wyjątki (checked i unchecked)
- Polimorfizm

Druga część to kalkulator w Javie. Cała rozmowa po angielsku, bo był gość z zagranicy. Kolejny etap miał być o system design."

Widzisz wzorzec? Najpierw podstawy teoretyczne, potem praktyka.

9.2. Rozmowa techniczna z algorytmami

Mój student Michał napisał:

"Miałem trzy zadania algorytmiczne:

1. Przekształcenie stringa - 'wwwegwwh' na '3w1e1g2w1h'
2. Liczba minut między godzinami - '11:00am-8:00pm' to 540 minut
3. Największy zysk z akcji na podstawie tablicy cen

Pytania nie były specjalnie trudne, ale presja czasu..."

Zauważ - te same zadania, które podałem wcześniej w tym kompendium!

9.3. TDD i Spock

Ola podzieliła się swoim doświadczeniem:

"Dostałam zadanie z TDD i Spock (Groovy). Trzeba było dodać do projektu banku opcję ustawienia waluty i wpłat w innej walucie z przeliczeniem.

Było stresująco - najpierw naprawa kodu z błędami, potem nowe funkcje. Spock był dodatkowym wyzwaniem, bo nie miałam z nim doświadczenia."

To ciekawy przykład, bo pokazuje, że firmy często patrzą na umiejętność pracy z istniejącym kodem, nie tylko pisanie od zera.

9.4. Rekrutacja Spring Boot z API zewnętrznym

"Moje zadanie to aplikacja Spring Boot komunikująca się z API pogodowym. Endpoint przyjmował współrzędne, zwracał dane pogodowe.

Aplikacja musiała też zapisywać w bazie informacje o każdym wywołaniu. Trzeba było znać Spring, REST API i pracę z zewnętrznymi serwisami."

Znowu - bardzo praktyczne zadanie, sprawdzające umiejętności, których faktycznie będziesz używać.

9.5. Pytania teoretyczne z rozmowy rekrutacyjnej

Karol opisał swoją rozmowę:

"Pytania były o:

- Różnice między INNER JOIN a LEFT/RIGHT JOIN
- Transakcje w bazach, @Transactional
- Primary key i foreign key
- SQL vs NoSQL
- Wzorce projektowe, głównie Singleton
- REST API best practices
- Metody HTTP
- PUT vs PATCH
- Adnotacje w Spring
- Git - merge vs rebase

Pokazywali też fragmenty kodu i pytali, co się stanie."

Co ciekawe, niemal wszyscy moi studenci wspominali pytania o bazach danych. To nie przypadek.

Podsumowanie

Wiesz co jest najciekawsze w tych wszystkich zadaniach? Nie sprawdzają one wcale, czy umiesz na pamięć definicję polimorfizmu. Sprawdzają, czy potrafisz rozwiązywać problemy.

Patrząc na te przykłady, widać wyraźnie, że firmy rekrutują ludzi, którzy będą w stanie tworzyć faktyczne rozwiązania, a nie recytować dokumentację.

Zebrałem dla Ciebie ponad 40 prawdziwych zadań rekrutacyjnych. Nie wymyśliłem ich - to autentyczne wyzwania, które moi podopieczni dostawali podczas rozmów.

Pamiętam, jak jeden z moich studentów powiedział: "Gdybym wiedział, że będą pytać o takie rzeczy, zupełnie inaczej bym się przygotowywał". I miał rację.

Większość kursów uczy Cię składni, a potem wypuszcza na rynek pracy kompletnie nieprzygotowanego na to, czego faktycznie oczekują pracodawcy.

W moich kursach prowadzę Cię krok po kroku przez wszystkie te zagadnienia. Budujemy razem solidne fundamenty, na których możesz później oprzeć swoją karierę.

Powodzenia w przygotowaniach! A gdybyś potrzebował dodatkowego wsparcia - wiesz, gdzie mnie znaleźć.

Bartek Kalka

Trener i mentor programistów Java - javaready.pl